校园奶茶店数据库应用第二阶段实验报告

第五周至第九周成果 详细图文版 v1 0

小组成员 袁远冬 陆乐天 罗金浩

课程 数据库系统实验 提交日期 2026 年 11 月 8 日

一 设计思路

本阶段从第一阶段奶茶店数据库出发，完成 ER 模型、函数依赖与范式分析，在保留业务数据的副本上迁移，并构建连接 SQL Server 的分角色 Web 应用。六张原业务表保留，增加账号及订单来源状态；顾客维护自己的草稿，店员确认时以事务扣减库存，管理员维护商品与库存。

本次验收中，迁移前后六表原数据、订单明细、商品销量与库存完全一致，完成销售仍为 6 单、15 杯、86.00 元；13 项 SQL 回归、43 例 API 与数据库及交叉场景集成测试、18 项真实浏览器流程均通过。测试证据和源文件随提交包提供。 本版进一步补充 21 个真实界面状态及 SQL 和测试输出截图，逐项说明执行动作、输入、观察位置与结论。

周次	任务与交付	主要位置
5	实体、标识、联系基数、参与约束与问题清单	docs/er-v01.svg、er-v10.svg、design.md
6	FD 与范式、异常与分解、数据迁移和对照	sql/migration.sql、regression.sql；result/before-after.json
7 至 9	分层 Web、登录 CRUD、角色、事务及异常	web/、tests/；result/浏览器截图及测试记录
阶段	操作手册、分工、AI 记录、可复现提交	README.md、docs/user-manual.md、contribution.md、ai_log.md

业务范围沿用校园单店、标准饮品和可售杯数，不扩展配方、糖冰加料、采购及支付退款。库存原本是开库盘点快照，迁移不得按历史销量再扣减。正文的设计结论来自实际表结构与业务规则，样例回归用于检查实现，不代替范式证明。

仓库 https://github.com/Ageha-yyd/campus-shop-db-lab

代码验收基线 87e6d9460421 第二阶段目录 stage2

第五周 原有设计的 ER 模型

v0.1 包含商品、库存、会员、员工、订单、订单明细六个实体。商品与订单的多对多联系通过明细关联实体实现。明细以订单号和行号联合标识，同一饮品可以在一张订单重复出现。

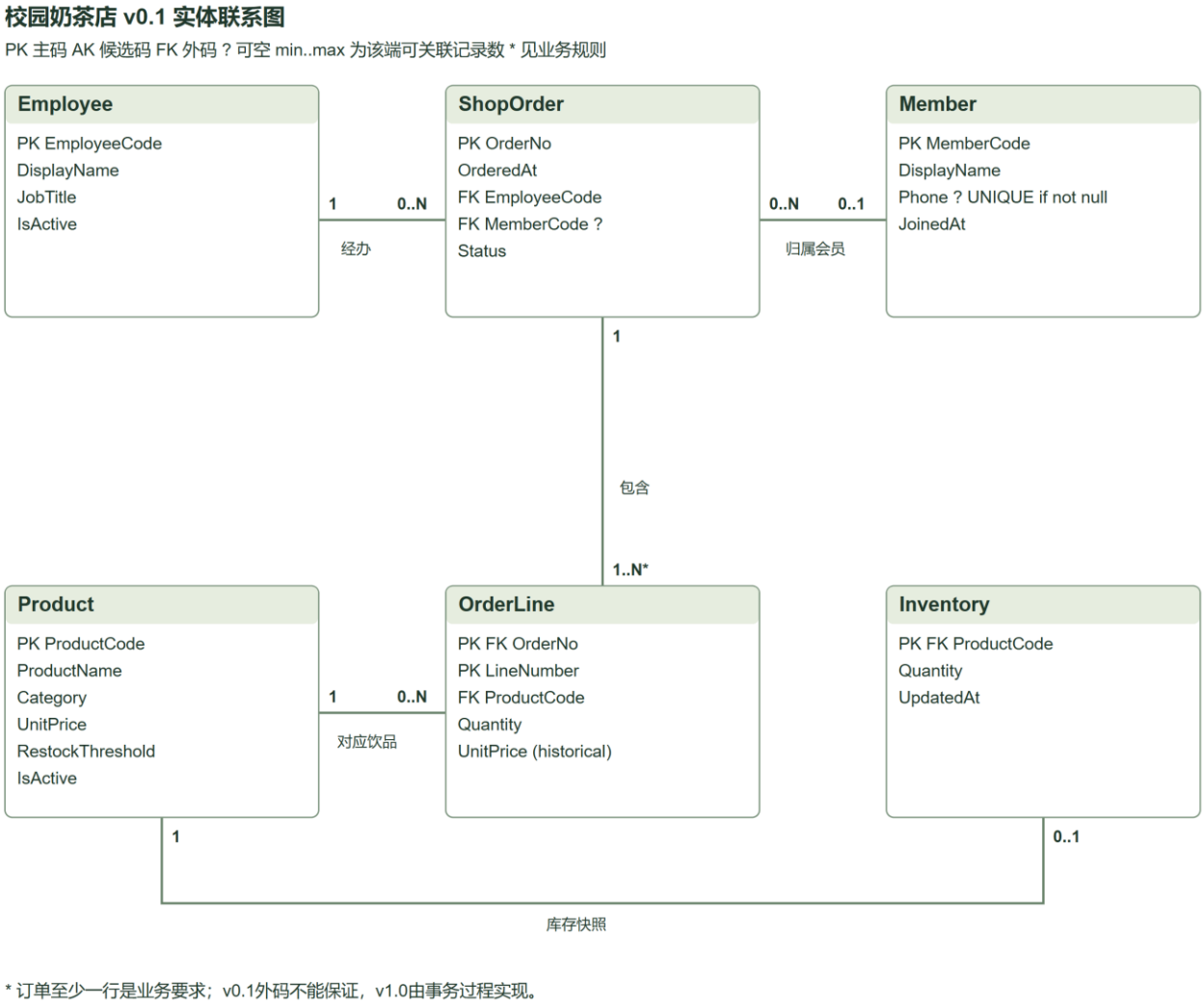


图 1 v0.1 ER 模型 可编辑源文件 docs/er-v01.svg

图中端点标注表示另一端单条记录所对应的此端记录数。0 表示可选参与，1 表示必须参与；PK 为主码、AK 为候选码、FK 为外码、问号表示可空。Inventory 共享商品主码，业务允许商品暂未登记库存，因此商品到库存为 0..1。

订单至少一条明细是业务规则；仅有 FK 不能保证这一最低基数。历史单的员工必填，会员可空；样例中的散客订单和多商品订单都与该规则一致。

第五周 目标设计的 ER 模型

v1.0 增加 AppUser；订单新增创建账号与扣库来源标志，允许草稿暂无员工。店员确认后必须写入员工；历史订单创建账号保留 NULL，不虚构来源。

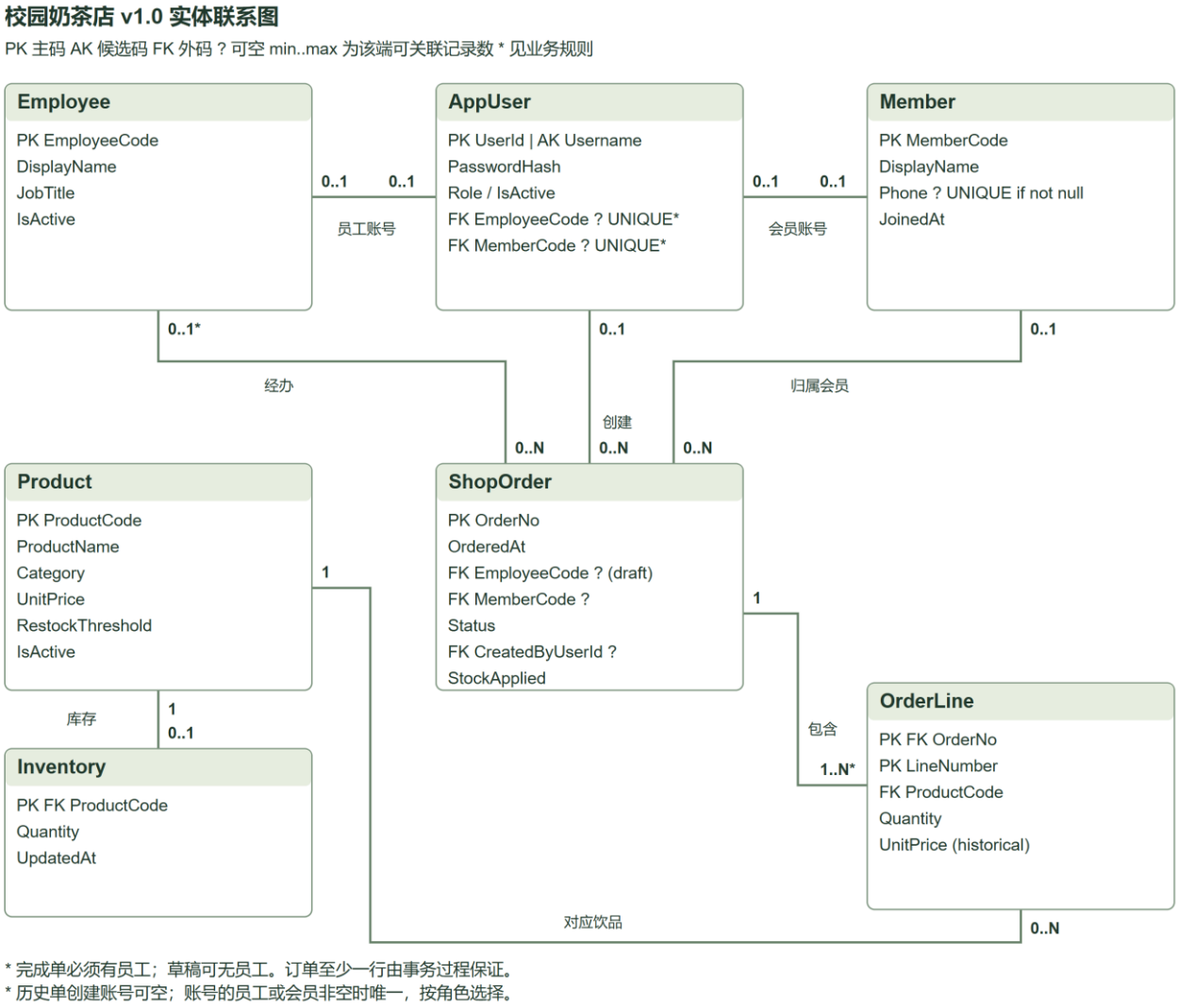


图 2 v1.0 ER 模型 可编辑源文件 docs/er-v10.svg

AppUser.UserId 为主码，Username 是另一个候选码；员工号与会员号只在非空子集唯一，不是全关系候选码。admin 和 clerk 必须绑定员工，customer 可以绑定会员或为散客，不能把两个可选联系误画成账号必须同时关联员工与会员。

SchemaVersion 用于迁移版本记录；Baseline_*保存迁移前查询和数据快照，不作为业务实体。字段类型、空值、默认值和全部索引约束由实际 SQL Server 目录导出到 data-dictionary.md。

第五周 业务规则与原设计改进

联系	两端业务基数	映射与执行方式
商品与库存	商品 0..1 库存；库存恰好 1 商品	Inventory.ProductCode 共享 PK/FK；新建商品自动建零库存，允许删零库存
商品与明细	商品 0..N 明细；明细恰好 1 商品	OrderLine.ProductCode FK；历史引用限制删除，可下架
订单与明细	订单 1..N 明细；明细恰好 1 订单	复合 PK、FK；最低一行由事务保存过程与确认过程检查
员工与订单	员工 0..N 订单；草稿 0..1 员工，完成单 1 员工	可空 FK；确认过程使用店员账号对应员工
会员与订单	会员 0..N 订单；订单 0..1 会员	可空 MemberCode；顾客会员归属在后端确定
账号与订单	账号 0..N 订单；订单 0..1 创建账号	CreatedByUserId FK；历史为空，新增必填；顾客按创建人过滤
账号与员工或会员	两端各 0..1，依角色选择	可空 FK 和过滤唯一索引；CHECK 限制员工/顾客配置

问题清单及保留理由

保留：六表避免重复存储会员名、员工名和商品信息；OrderLine 的历史成交价不依赖当前菜单价；订单金额由明细聚合而不重复存储。Category 只有原子类别名称，无类别编号或其他类别属性，不为展示拆表而破坏现有关系。

改进：原库没有登录账号，库存不自动扣减，订单没有草稿状态；外码也不能保证头与明细同时写入。增加 AppUser 和 DRAFT 状态，以受限过程保存非空草稿，确认时统一扣库存。StockApplied 区分新系统实际扣库与历史开库订单，避免错误回补。

验证映射：会员 customer 绑定 M001；guest 的 MemberCode 为空；浏览器创建 D001 与 D003 的多商品草稿，店员确认后 D001 库存 20→17、D003 库存 5→4。该流程将图中的可选会员、订单关联明细与商品库存实际连通。

第六周 逐表函数依赖与范式判断

以下依赖来自本项目业务规则。“其余字段”指同一关系内字段。全部字段为原子值，满足 1NF；判断 2NF 时考虑所有候选码，判断 3NF 时检查非码决定因素。

表与候选码	关键函数依赖	范式及理由
Product ProductCode	ProductCode → 名称 类别 当前价 阈值 启用状态	BCNF；无已知非码决定因素；名称和类别不唯一
Inventory ProductCode	ProductCode → Quantity UpdatedAt	BCNF；可售杯数与维护时间依赖商品主码
Member MemberCode	MemberCode → 名称 电话 加入日期	BCNF；可空电话不是全关系候选码，仅非空子集唯一
Employee EmployeeCode	EmployeeCode → 名称 职称 启用状态	BCNF；同职称或姓名不能决定员工号
ShopOrder OrderNo	OrderNo → 时间 员工 会员 状态 创建账号 扣库来源	BCNF；Status 不能单独决定 StockApplied，历史与新增完成单不同
OrderLine (OrderNo,LineNumber)	(OrderNo,LineNumber) → ProductCode Quantity UnitPrice	BCNF；非主属性依赖完整复合码；订单号或商品号单独不决定成交价
AppUser UserId 与 Username	UserId → 其余字段；Username → 其余字段	在声明全域 FD 下 BCNF；两个候选码均为决定因素，Role 不决定员工或会员编号

对单属性候选码，非主属性不存在对码的部分依赖；OrderLine 中订单号并不决定商品、杯数和价格，行号在不同订单中也不能单独决定明细，因而满足 2NF。没有将商品名、会员名、员工名从其他实体复制到订单中，避免非码的传递依赖。

条件唯一、可空、角色互斥与最小参与约束仍需单独实现；不能仅凭加入 UserId 自增主键，就宣称用户名等其他候选码的依赖已经消失。

第六周 更新异常与分解依据

实际关系的插入修改删除分析

对象	如何避免异常
商品与库存	商品可独立存在；盘点只更新一条库存；删除零库存不删除商品。被历史明细引用的商品不能误删
会员与员工	各自可以独立登记；名称只维护一处；历史订单通过外码引用，限制破坏性删除
订单与明细	订单属性只存订单头；明细保存商品与成交价。头和非空明细在同一事务保存，完成单保留历史
账号	唯一登录名和独立密码哈希；员工与会员名称不复制到账号。禁用账号后已有会话也被拒绝

独立异常关系及无损证明

演示关系 R(ProductCode,ProductName,CategoryCode,CategoryName)，F={ProductCode→ProductName,CategoryCode；CategoryCode→CategoryName}。P01 与 P02 同属 C01 奶茶，只把 P01 类别名改为鲜奶茶就产生同编号不同名称的修改异常；无商品时无法登记新类别是插入异常；删除某类最后商品会丢失类别信息是删除异常。

分解为 P(ProductCode,ProductName,CategoryCode)与 C(CategoryCode,CategoryName)。交集{CategoryCode}是 C 的码，(P∩C)→C 成立，因此二元分解无损；原两条关键依赖分别保留在 P 和 C 中，保持依赖。normalization_demo.sql 真实复现修改异常、连接还原与独立类别登记；有限样例通过只作辅助检查。

实际 v0.1 没有 CategoryCode，也没有类别的其他属性，不能凭假设关系指认原表存在该传递依赖，因此六张核心关系不做拆分。OrderLine.UnitPrice 是当次成交事实，Product.UnitPrice 是当前价格，两者含义不同；历史价保留不属于不当冗余。订单总额由明细聚合；StockApplied 由确认和取消事务维护。

二 实验过程

副本迁移与失败恢复

首先核对 GitHub 主分支与本地基线 7b77969 一致。默认初始化对原 MilkTeaShopV01 做 COPY_ONLY、CHECKSUM 备份，用独立文件名还原到新的目标库。目标已存在或.env 已存在时直接拒绝覆盖。原库不删、不重建、不重命名；失败调试副本也保留。

顺序	执行内容与实际结论
1 保存基线	检查商品 6 行、订单 7 行、重复明细及空订单；保存六表及订单详情/销量/库存快照
2 迁移结构	同一事务增加 AppUser、SchemaVersion 与唯一索引；订单增加创建账号与 StockApplied，扩展 DRAFT 并调整员工可空及默认状态
3 配置应用	更新详情视图为可显示无员工草稿；创建受限订单过程与权限；配置四个密码应用角色及四个哈希登录账号
4 回归对照	原有六表仍为 6/6/4/3/7/11 行；双向 EXCEPT 比较详情、销量、库存；校验金额、键、约束及权限
5 空环境验证	--from-scripts 从仓库自带 v0.1 结构与样例建立独立基线，随后执行同一迁移，不以删库重建代替已有数据迁移

migration.sql 使用 XACT_ABORT 及单事务。结构或数据冲突抛错后回滚；不自动合并重复编号，也不凭空补造订单明细。SchemaVersion 存在即拒绝再次迁移。配置后半段出错时保留失败副本，修复后换新目标名。

迁移后需退回时，停止应用并指回保留的 v0.1 库或还原备份；先单独保存 v1.0 新增订单，不宣称降级能够无损接纳新结构的数据。新增 AppUser 四行、版本记录一行和 Baseline_*快照是辅助数据增加，不是业务数据重复或丢失。

证据位置：result/setup.json、before-after.json、regression.txt。原库、迁移快照及目标六表原字段的 SHA256 与逐行比较均一致。

迁移前后关键结果与 SQL 回归

项目	迁移前预期	迁移后实际	结论
六业务表行数	6 / 6 / 4 / 3 / 7 / 11	6 / 6 / 4 / 3 / 7 / 11	一致
订单详情	11 条明细及全部字段	双向 EXCEPT 均无差集	一致
完成订单	6 张	6 张	一致
完成销量与金额	15 杯 / 86.00 元	15 杯 / 86.00 元	一致
D001 至 D006 销量	4 / 3 / 5 / 3 / 0 / 0	4 / 3 / 5 / 3 / 0 / 0	一致
D001 至 D006 库存	20 / 12 / 5 / 0 / 18 / 9	20 / 12 / 5 / 0 / 18 / 9	一致
历史扣库标记	全部 0；不重复扣盘点库存	全部 0	一致

回归项	实际结果
01 至 06 语义与标识	明细、销量、库存双向相等；金额/数量正确；商品会员员工原属性保留；历史单无重复扣库
07 至 10 完整性与价格	不存在订单的外码、负库存被 547 拒绝；合法商品与库存写入成功后回滚；修改当前价格不改历史成交价
11 至 13 权限	店员可读菜单；改价与读取账号哈希均以 229 拒绝；临时测试用户与写入全部回滚

result/before-after.json 逐项保存 expected 和 actual，不只保留成功截图。取消订单不计销售，零销量 D005/D006 仍返回 0。商品当前菜单价与历史成交价分开验证，避免误把两种价格强制同步。

独立规范化演示也已实际运行，记录在 normalization-demo.txt。它只使用临时表，不更改业务结构。

分层实现与事务安全

层级	文件与职责
页面层	templates/index.html、static/app.js 与 app.py 路由：表单交互、fetch API、接收输入与展示统一结果
业务层	service.py：输入范围、角色、所有权与状态规则；组织草稿保存、确认与取消流程
数据访问层	dao.py：唯一运行时 SQL 入口；?参数绑定；连接打开、事务提交/回滚、finally 关闭
数据库层	SQL Server 表、CHECK/UNIQUE/FK、视图与订单过程；锁定订单并维护库存一致性

查询路径：菜单搜索 GET→Service.products→DAO.products→绑定搜索参数查询 Product→统一 JSON→DOM 的 textContent 展示。修改路径：确认 POST→业务角色/可见性检查→DAO 事务→usp_OrderAction 锁定订单、按商品号汇总和锁库存、校验可售杯数、扣库并完成→提交。任何中途错误都 rollback，连接最终释放。

同商品多行先汇总，再一次性扣减。重复确认被状态限制拒绝；v1.0 完成单取消仅回补一次。真实并发用例中，两单各需 D001 十五杯、库存二十杯，结果一单 200、一单 409，剩五杯。第二商品缺货时，第一商品已经执行的扣减也回滚。

权限与统一异常

数据库连接以 Windows 身份建立后，立即通过顶层 ODBC CALL 激活密码应用角色，再执行任何业务 SQL；禁用连接池，不申请角色恢复 cookie，关闭后销毁。四个有效身份均实测 sysadmin=0、db_owner=0；店员/顾客不能直接改价、改库存、读账号哈希或订单全表。顾客个人所有权还由服务会话检查。

角色	可执行范围	关键限制
顾客含会员和散客	菜单库存查询；维护自己的非空草稿与明细，查看确认结果	不能确认扣库、改商品/库存、查销量或其他顾客订单
店员	管理全部草稿、确认扣库、取消新完成单并回补、查库存销量	不能改菜单或直接盘点库存；历史开库单不可按新流水回补
管理员	商品和库存维护；同时具备店员订单操作	被引用商品不可硬删；无架构、账号管理及数据库所有者权限

统一响应 ok/code/message/data/request_id；区分参数、认证、权限、资源、业务、连接、约束和事务错误。界面不暴露 SQL、连接、路径或堆栈；后端 JSONL 记录时间、请求编号、操作、角色、错误码与结果。CSRF、HttpOnly/SameSite 会话、scrypt 哈希和账号有效性检查同步启用。

小组实施顺序与实验环境

本报告按需求分析、模型论证、迁移实现、应用实现、运行验证和复核整改组织，展示项目实际形成的过程。代码、SQL 和自动化运行由 AI 工具辅助完成；组员责任及后续演示安排列于末尾，不把自动化记录写成未经确认的个人操作或人工签字。

从任务到实现的过程

第一步，对照第五周、第六周和第 7 至 9 周任务讲解，把交付拆成模型、规范化、迁移和 Web 四部分。先确认校园单店及可售杯数的业务范围，读取 v0.1 六表及历史样例，列出保留项和缺口，避免在不存在的类别编号上杜撰传递依赖。

第二步，从订单和明细的业务事实确定键与联系：一次购买可以有多种饮品，会员可选，明细必须属于订单；再按草稿、确认、取消的状态补充账号和库存来源。每个基数都明确由主外码、可空、唯一约束或事务过程中的哪一层执行。

第三步，逐表分析函数依赖和全部候选码。六张原表已有合理分解，保留结构和历史成交价；另用临时关系演示异常、无损分解和保持依赖，而非为增加表数改坏原设计。

第四步，先建立独立副本，再迁移、保存 Baseline 快照、安装受限过程和应用角色。用双向 EXCEPT、行数和原字段摘要对照原数据，确认 6 笔完成单、15 杯、86.00 元及库存均未改变，再接入页面。

第五步，按页面、路由、业务、DAO 和 SQL 分层实现登录及 CRUD。先验证一条查询和一次草稿写入，再串联顾客下单、店员确认、库存和销量；最后覆盖越权、缺货回滚、并发、取消和连接故障。复核暴露的五项缺陷修复后重新执行全部验收。

项目	本次实测配置
运行环境	Windows；Python 3.14；SQL Server 2025 Express 17.0.1000.7；ODBC Driver 18
应用与浏览器	Flask 3.1.2；pyodbc 5.3.0；pytest 8.4.2；Playwright 驱动真实 Microsoft Edge
原库与演示库	MilkTeaShopV01 保留；演示副本 MilkTeaShopV10_Final
截图与集成验收库	MilkTeaShopV10_Acceptance_Test；只清理明确以_Test 结尾的隔离测试库
运行方式	python -m stage2；复制后的启动第二阶段.bat 亦已验证；默认仅监听本机

关键字段与数据库约束的实现

模型转换为关系后，字段类型和空值约束必须能表达业务范围。下表列出应用链条中的关键字段；完整七表、默认值、索引、CHECK 和实际样例见 docs/data-dictionary.md，该字典从 SQL Server 目录导出。

关系及字段	类型或标识	设计作用与约束
Product.ProductCode	varchar(12) PK	稳定商品编号；名称不唯一，不作为码
Product.UnitPrice	decimal(10,2)	当前菜单价；服务限制 0.01 至 99999999.99，最多两位小数
Inventory.ProductCode	PK 兼 FK	每商品最多一条库存；可无登记；不重复商品名称
Inventory.Quantity	int	非负可售杯数；应用盘点上限 1000000，零杯才可删除记录
ShopOrder.OrderNo	varchar(24) PK	新增为 WEB 加随机标识；历史编号保持原样
ShopOrder.Status	varchar(12)	DRAFT、COMPLETED、CANCELLED；动作过程控制转换
EmployeeCode 与 MemberCode	可空 FK	草稿暂无员工；会员可空；确认时填写店员对应员工
CreatedByUserId	可空 FK 至 AppUser	历史来源为空；新增保存实际登录账号，用于顾客订单隔离
StockApplied	bit	新系统实际扣库后为 1，取消回补后为 0；历史开库订单为 0
OrderLine 联合标识	PK(OrderNo,LineNumber)	Quantity 与 UnitPrice 依赖完整联合码；同商品允许多行
OrderLine.UnitPrice	decimal(10,2)	保存草稿时的成交报价；确认不按新菜单价格重算
AppUser 两个候选码	UserId PK；Username UNIQUE	Role 用于权限，PasswordHash 为 scrypt 哈希；不向页面导出哈希

数据库完整性分为域、实体、参照和业务过程四部分。CHECK 拒绝负库存，PK/UNIQUE 拒绝重复标识，FK 限制不存在的引用；至少一条明细、库存是否足够、重复确认和取消仅回补一次，需由同一事务内的过程执行。只画 ER 或只在 HTML 设置输入范围均不足以保证这些规则。

金额字段区分单价与聚合总额：单价上限来自 decimal(10,2)，订单汇总使用 decimal(18,2)。复核已验证两行各 100 杯且单价 99999999.99 时，19999999998.00 元仍可正常读取。此为合法边界测试，并非常规菜单价格。

登录与查询在各层如何衔接

登录的输入 处理和输出

浏览器先 GET /api/session 取得 CSRF；登录表单把 username 和 password 以 JSON 发送到 POST /api/login。路由解析对象，Service 检查长度并调用 DAO，DAO 以用户名参数绑定查询启用账号；Service 用 scrypt 验证哈希。成功后清空旧会话，保存用户名和角色并更新 CSRF，页面只收到用户名与角色。错误密码统一 401，不暴露账号是否存在。

```
    def user(self, username):
        with self.transaction('auth') as conn:
            rows = self.rows(conn.execute('SELECT
UserId,Username,PasswordHash,Role,EmployeeCode,MemberCode FROM dbo.AppUser WHERE Username=? AND
IsActive=1', username))
            return rows[0] if rows else None

    def products(self, role, search=''):
        with self.transaction(role) as c:
            return self.rows(c.execute('SELECT
ProductCode,ProductName,Category,UnitPrice,RestockThreshold,IsActive FROM dbo.Product WHERE
ProductName LIKE ? OR ProductCode LIKE ? ORDER BY ProductCode', '%'+search+'%', '%'+search+'%'))
```

代码来源 web/dao.py 第 46 至 53 行；正文仅节选，完整文件随包提供。

上面用户名 SQL 的问号和后续 username 参数相对应；搜索语句有两个问号，分别绑定包含通配符的搜索值。SQL 文本的结构固定，输入不会被拼接成条件或多条语句。运行代码中的全部 ODBC 和业务 SQL 只位于 dao.py，service.py 不直接访问数据库。

菜单查询示例

输入 D001 时，app.js 发起 GET /api/products?q=D001；路由传递 q；Service 检查登录及搜索长度不超过 80；DAO 在商品名称和编号上绑定两个 LIKE 参数；数据库返回有序商品记录；统一 JSON 交给前端。前端用 textContent 放置名称、金额等值，避免把商品名称当作 HTML 执行。

搜索列表和订单编辑使用的完整商品目录分别维护。否则先搜 D001 后编辑含 D003 的草稿，会把未匹配商品从下拉框移走。该缺陷曾被实际复现，修复后已加入交叉场景回归。

位置	输入	输出或副作用
index.html 与 app.js	表单、按钮、搜索文字	fetch JSON；绘制菜单和通知；不执行 SQL
app.py	请求路径、JSON、会话、CSRF	解析与统一响应；生成 request_id；每次重查账号有效性
service.py	可信当前用户和清洗后的数据	校验角色、输入、所有权；调用 DAO
dao.py 与 SQL Server	绑定参数及固定角色	打开连接、激活应用角色、查询或事务、关闭连接

订单事务和库存一致性的实现

顾客草稿保存把非空 items 数组提交到服务层。后端要求 1 至 50 行、每行 1 至 100 杯，并拒绝布尔值或小数充当整数。保存过程校验商品存在且启用，确定实际账号和会员关联，在同一事务中写订单头与全部明细；修改时整体替换草稿明细并按当前菜单重新报价。草稿保存本身不扣库存。

```
    @contextmanager
    def transaction(self, role):
        conn = self.connect(role)
        try:
            conn.execute('SET XACT_ABORT ON; SET ANSI_WARNINGS ON; SET ARITHABORT ON; SET
NUMERIC_ROUNDABORT OFF;')
            yield conn
            conn.commit()
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()
```

代码来源 web/dao.py 第 29 至 40 行；正文仅节选，完整文件随包提供。

DAO 的事务上下文在 yield 之后统一 commit，异常时 rollback，finally 始终 close。连接建立后先激活固定白名单内的应用角色，之后才执行 SQL；禁用连接池，避免已激活角色的连接进入别的请求。SQL 过程检查与更新在同一个 DAO 事务中完成。

确认订单的数据库步骤

1 店员点击确认；服务层先校验可见性和 admin/clerk 角色。2 usp_OrderAction 对订单使用 UPDLOCK 和 HOLDLOCK，拒绝非 DRAFT 状态。3 将相同商品的多行数量汇总，按商品编号稳定顺序锁定商品与库存。4 每种商品先检查库存足够再扣减；任何一项缺货抛错，已执行的前项扣减也回滚。5 全部成功后写 COMPLETED、StockApplied=1 及当前店员，统一提交。

动作	允许的起始状态	实际更新与限制
保存或编辑	新草稿或自己的 DRAFT	原子保存头和明细；顾客不能伪造其他会员
确认	DRAFT；仅员工角色	库存扣减一次；状态完成；设置员工和扣库标记
取消草稿	DRAFT	状态取消；未扣库不回补
取消新增完成单	COMPLETED 且 StockApplied=1；员工	回补原数量，缺失库存行重建；标记归零；再次取消拒绝
删除	DRAFT	同事务删明细和头；完成单不能直接删除
取消历史完成单	历史 StockApplied=0	拒绝；历史销售已经包含在开库盘点中，不能凭历史单回补

服务端权限 异常处理和日志追踪

角色判断同时落在可信后端和有效数据库身份。隐藏维护按钮只改善操作界面；直接调用 API 仍须经过 Service.allow。DAO 用 Windows 认证建连接后立即激活 mt_auth、mt_admin、mt_clerk 或 mt_customer，再执行业务 SQL；实际测试四个有效身份均非 sysadmin、非 db_owner，越权 SQL 真实被拒绝。

应用角色服务于可信后端，不作为每个顾客独立的数据库登录。顾客所有权由后端会话的 UserId 传入受限订单过程；角色密码不交给顾客。店员和顾客没有原表任意写权限，管理员也没有架构和账号管理权限。

场景	统一状态和错误码	实际处理
错误凭据或未登录	401 AUTH_FAILED / UNAUTHENTICATED	不进入工作区；不暴露账号和哈希
越权或 CSRF 错误	403 FORBIDDEN / CSRF_INVALID	在服务端拒绝；非 ASCII 错误令牌也返回 403
不可见或不存在订单	404 NOT_FOUND	统一说明不可访问，避免跨顾客枚举
无效输入	400 INPUT_INVALID	服务端类型、范围和空数组检查
唯一和 FK 冲突	409 CONSTRAINT_CONFLICT	数据库原始异常转为友好提示，事务回滚
缺货和错误状态	409 BUSINESS_RULE	不部分扣库；重复确认及取消受限
真实连接故障	503 DB_UNAVAILABLE	不向界面返回连接地址、SQL 或堆栈
事务失败分类	503 TRANSACTION_FAILED	40001 类错误测试采用明确故障注入，验证分类及日志

本轮 W11 实际请求的公开日志节选如下。界面中的编号与 request_id 相同；按编号即可定位本机详细原因，公开记录只导出安全字段。


```
{
 "time": "2026-11-08T10:20:14.555437+00:00",
 "request_id": "ed17929468424de4ab09337c84c8d935",
 "operation": "POST /api/products",
 "role": "admin",
 "code": "CONSTRAINT_CONFLICT",
 "result": "failure",
 "http_status": 409,
 "duration_ms": 373
}
```


原始公开证据 result/detailed-request-log.jsonl；每条 HTTP 记录可与 detailed-browser.json 按 request_id 对应。

运行证据的读取方式与复现实验步骤

以下每个步骤编号均绑定一次实际执行或实际界面状态。W 为真实 Edge 操作，S 为 SQLCMD 执行，T 为 pytest 执行。图 1 和图 2 是模型设计图；后续业务界面均为真实运行截图或其连续矩形裁剪，未改写图中金额、状态和提示。SQL 及测试输出在只读核验页展示，页面明确注明来源，不冒充 SSMS 或终端。

编号	执行动作	对应输入和证据
准备	创建独立_Test 库并配置 test.env	setup 从随包 SQL 建立基线，再执行真实迁移；禁止覆盖现有库
T01	执行 pytest stage2/tests -v	43 个参数化及交叉用例；XML 与完整 stdout 随包提供
S01	SQLCMD 运行 report_verification.sql	查状态计数、完成单总额、六种商品库存销量、11 条明细
S02	SQLCMD 运行 regression.sql	双向语义对比、完整性与权限，共 13 个 PASS；临时写入回滚
S03	SQLCMD 运行 normalization_demo.sql	仅临时表；复现修改异常，再分解、连接和独立登记类别
W01 至 W21	运行 browser_acceptance --detailed	四账号在真实 Edge 执行表单、CRUD、确认、取消和故障；21 状态截图
收尾	再次验证基线并核对源数据	只清理_Test 库临时订单和商品；原库与演示库保持保留

首次环境按 stage2/README.md 创建虚拟环境和独立数据库。已配置环境的采集命令：

```
.\.venv\Scripts\python.exe -m stage2.tools.capture_report_evidence
.\.venv\Scripts\python.exe -m stage2.tools.browser_acceptance --detailed
.\.venv\Scripts\python.exe -m pytest stage2/tests/test_integration.py::test_baseline_and_migration -q
```

上表按建议复现和阅读顺序编排，实际执行时间以 UTC 记录为准；本轮集成输出在界面采集后复跑，用例清理同时恢复基线。W 图属于同一个界面流程；草稿编号以图中 WEB 编号为准，后续修改、确认、取消均是该编号。不同实验轮次会生成新编号，不应把两个轮次的图当作同一订单。

执行输出的核验页逐行呈现 subprocess 实际输出，注明命令、执行时间、退出码及输出 SHA256；截图与原始文本的对应行号在 result/detailed-runs/manifest.json 中可查。

S03 规范化异常与分解的实际执行

步骤 S03：SQLCMD 运行 normalization_demo.sql。在临时关系 R 中插入 P01、P02，两者均属于 C01 奶茶；只把 P01 的类别名称改为鲜奶茶。

观察中间结果，同一个 CategoryCode=C01 出现两个名称，实际复现修改异常。随后分成商品关系与类别关系，只修改类别表的一行；末尾连接结果两行都为鲜奶茶，且 PASS 说明原关系还原与独立类别登记检查通过。无损与保持依赖的理论证明见前文，样例只是实现核对。

规范化异常和分解演示 实际输出 1/1

这是执行输出的只读核验页，非SSMS或终端原生界面。原文逐行保留。

数据库 MilkTeaShopV10_Acceptance_Test　退出码 0　执行开始 UTC 2026-11-08T10:21:41.250525+00:00

执行命令 sqlcmd -S .\SQLEXPRESS -d MilkTeaShopV10_Acceptance_Test -E -C -b -f 65001 -W -w 180 -i D:\Homework\数据库\campus-shop-db-lab\stage2\sql\normalization_demo.sql

```
Step ProductCode ProductName CategoryCode CategoryName
---- ----------- ----------- ------------ ------------
修改前 同类别重复两行 P01 珍珠奶茶 C01 奶茶
修改前 同类别重复两行 P02 椰果奶茶 C01 奶茶
Step ProductCode ProductName CategoryCode CategoryName
---- ----------- ----------- ------------ ------------
只改一行产生不一致 P01 珍珠奶茶 C01 鲜奶茶
只改一行产生不一致 P02 椰果奶茶 C01 奶茶
ProductCode ProductName CategoryCode CategoryName
----------- ----------- ------------ ------------
P01 珍珠奶茶 C01 鲜奶茶
P02 椰果奶茶 C01 鲜奶茶
PASS anomaly reproduced; decomposition roundtrip equal; standalone category insert and single-row rename succeed
```

原始输出 result/detailed-runs/S03_normalization.txt　SHA256 fdcaa6ea9162606f734271a017ee7a1ab46cb4bc4e1401317d1ae4630dce19be
本页为原输出第 1 至 13 行

图 3 S03_normalization 规范化异常和分解演示 输出第 1 至 13 行

原始证据 result/detailed-screenshots/S03_normalization-1.png

S01 基线金额 库存与销量的实际核验

步骤 S01：运行 sql/report_verification.sql；先按 Status 分组，再对 COMPLETED 订单关联明细求和，并查询六种商品当前库存、销量和销售额。

读图先看库名确认是隔离_Test 库，再看 COMPLETED=6、CANCELLED=1 和 6 单 15 杯 86.0000 元。D001 库存 20、销量 4；D003 库存 5、销量 5；D005 与 D006 销量 0 仍列出。库存是盘点快照，不应再减去历史销售量。



SQL基线和业务数值核验 实际输出 1/2

这是执行输出的只读核验页，非SSMS或终端原生界面。原文逐行保留。

数据库 MilkTeaShopV10_Acceptance_Test　退出码 0　执行开始 UTC 2026-11-08T10:21:40.350530+00:00

执行命令 sqlcmd -S .\SQLEXPRESS -d MilkTeaShopV10_Acceptance_Test -E -C -b -f 65001 -W -w 180 -i D:\Homework\数据库\campus-shop-db-lab\stage2\sql\report_verification.sql

```
DatabaseName
------------
MilkTeaShopV10_Acceptance_Test
Status Orders
------ ------
CANCELLED 1
COMPLETED 6
CompletedOrders Cups Amount
--------------- ---- ------
6 15 86.0000
ProductCode ProductName Quantity UnitsSold SalesAmount
----------- ----------- -------- --------- -----------
D001 珍珠奶茶 20 4 24.00
D002 满杯百香果 12 3 21.00
D003 冰鲜柠檬水 5 5 20.00
D004 椰果奶茶 0 3 21.00
D005 柠檬红茶 18 0 .00
D006 茉莉绿茶 9 0 .00
```

原始输出 result/detailed-runs/S01_baseline.txt　SHA256 e56adf0195bf8d463cbce8ff94468093c2b63aa99a06ef58906a95503ee89495
本页为原输出第 1 至 18 行

图 4 S01_baseline SQL 基线和业务数值核验 输出第 1 至 18 行

原始证据 result/detailed-screenshots/S01_baseline-1.png

S01 十一条历史订单明细的实际输出

继续步骤 S01：按 OrderNo、LineNumber 排序查询 OrderLine，查看编号、行号、商品、杯数和历史成交单价。

图中保留 11 条明细，其中取消订单的明细仍存在；销售汇总按订单状态过滤，不能把所有明细都算入完成销售。MT-20261001-001 包含 D001 两杯和 D002 一杯，验证一张订单可包含多商品。逐行迁移前后 expected/actual 见 before-after.json，13 项回归再做双向比较。

SQL基线和业务数值核验 实际输出 2/2

这是执行输出的只读核验页，非SSMS或终端原生界面。原文逐行保留。

数据库 MilkTeaShopV10_Acceptance_Test　退出码 0　执行开始 UTC 2026-11-08T10:21:40.350530+00:00

执行命令 sqlcmd -S .\SQLEXPRESS -d MilkTeaShopV10_Acceptance_Test -E -C -b -f 65001 -W -w 180 -i D:\Homework\数据库\campus-shop-db-lab\stage2\sql\report_verification.sql

```
OrderNo LineNumber ProductCode Quantity UnitPrice
------- ---------- ----------- -------- ---------
MT-20261001-001 1 D001 2 6.00
MT-20261001-001 2 D002 1 7.00
MT-20261001-002 1 D003 3 4.00
MT-20261002-001 1 D003 1 4.00
MT-20261002-002 1 D005 1 5.00
MT-20261003-001 1 D004 2 7.00
MT-20261003-001 2 D001 1 6.00
MT-20261004-001 1 D002 2 7.00
MT-20261004-001 2 D004 1 7.00
MT-20261004-002 1 D003 1 4.00
MT-20261004-002 2 D001 1 6.00
```

原始输出 result/detailed-runs/S01_baseline.txt　SHA256 e56adf0195bf8d463cbce8ff94468093c2b63aa99a06ef58906a95503ee89495
本页为原输出第 19 至 31 行

图 5 S01_baseline SQL 基线和业务数值核验 输出第 19 至 31 行

原始证据 result/detailed-screenshots/S01_baseline-2.png

S02 迁移 完整性和权限的实际回归

步骤 S02：SQLCMD 以-b 遇错返回非零，运行 regression.sql。每项先执行断言，满足后才打印对应 PASS；任一不相等或非法写入被接受，脚本会抛错。

图中 01 至 06 核对原数据语义与历史库存；07、08 以 547 拒绝非法外码和负库存；09 说明当前菜单改价不改历史价；10 合法写入成功后回滚；11 至 13 验证店员可读菜单但不能改价或读账号凭据，拒绝码 229。共 13 个 PASS、退出码 0，原始完整输出也保留末尾业务结果表。



迁移与权限SQL回归 实际输出 1/3

这是执行输出的只读核验页，非SSMS或终端原生界面。原文逐行保留。

数据库 MilkTeaShopV10_Acceptance_Test　退出码 0　执行开始 UTC 2026-11-08T10:21:40.634623+00:00

执行命令 sqlcmd -S .\SQLEXPRESS -d MilkTeaShopV10_Acceptance_Test -E -C -b -f 65001 -W -w 180 -i D:\Homework\数据库\campus-shop-db-lab\stage2\sql\regression.sql

```
PASS 01 exact order detail equality both directions
PASS 02 product sales equality including zero sales
PASS 03 opening inventory equality
PASS 04 completed totals 6 orders / 15 cups / 86.00
PASS 05 business identities and profiles preserved
PASS 06 historical stock not deducted again
PASS 07 invalid foreign key rejected 547
PASS 08 negative stock rejected 547
PASS 09 current menu price independent of historical price
PASS 10 valid inserts accepted and rolled back
PASS 11 clerk SELECT permitted
PASS 12 clerk price write denied 229
PASS 13 credential read denied 229; temporary test user rolled back
ProductCode ProductName UnitsSold SalesAmount
----------- ----------- --------- -----------
D001 珍珠奶茶 4 24.00
D002 满杯百香果 3 21.00
D003 冰鲜柠檬水 5 20.00
```

原始输出 result/detailed-runs/S02_regression.txt　SHA256 a5940bb4e3b36c2d83d9b07db5c26422a733db194a354c350d4199d20562844e
本页为原输出第 1 至 18 行

图 6 S02_regression 迁移与权限 SQL 回归 输出第 1 至 18 行

原始证据 result/detailed-screenshots/S02_regression-1.png

W01 W02 登录操作和错误密码反馈

步骤 W01 访问 127.0.0.1:5001，尚未提交凭据。观察结果：显示登录表单；不展示密码。从本机私有账号文件读取各自随机密码；报告和提交包不包含密码。



图 7 W01 登录界面

原始证据 result/detailed-screenshots/D01_login.png

步骤 W02 customer 输入错误密码后提交。观察结果：用户名或密码错误；请求返回 401；停留登录页。HTTP 记录为 401 AUTH_FAILED；错误密码不会建立登录会话。

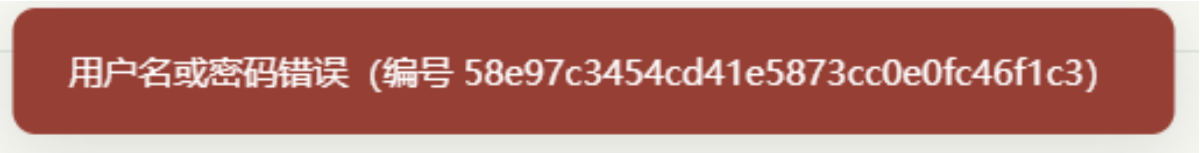


图 8 W02 错误密码反馈

原始证据 result/detailed-screenshots/D02_wrong_login.png

两图分别对应提交前的登录表单和提交错误密码后的实际红色通知。错误提示下附本次请求编号，方便从日志定位；不能把该图当作成功登录的证据。正确登录见下一步骤。

W03 顾客登录后读取实际商品菜单

步骤 W03 使用 customer 正确凭据登录。观察结果：六种商品；顾客无商品维护和销售入口。登录成功后的用户身份、菜单名称与价格均由后端返回；D001 为 6.00 元、D003 为 4.00 元。截图包含角色入口和六种商品。



图 9 W03 顾客菜单与角色入口

原始证据 result/detailed-screenshots/D03_customer_menu.png

检查要点：customer 可以切换菜单、订单和库存；销售统计及管理员维护表单不可见。入口差异与后端权限测试配合使用，后面的越权 API 用例才是权限拒绝的直接证据。

W04 W05 多商品草稿的新增和修改

步骤 W04 D001 两杯、D003 一杯，点击保存草稿。观察结果：订单 WEB-acd02e7cd4894de4ab4c；待确认；2×6+1×4=16.00 元。订单头与两条明细一次保存成功；此时仍为 DRAFT，不扣库存。



图 10 W04 保存多商品会员草稿

原始证据 result/detailed-screenshots/D04_draft.png

步骤 W05 编辑同一订单，把 D001 数量从 2 改为 3，再保存。观察结果：同一订单；3×6+1×4=22.00 元。两图订单编号相同。D001 数量 2 到 3，金额 12 到 18；D003 仍为 1 杯 4 元，所以总额 16 到 22 元。



图 11 W05 修改草稿并重新计算

原始证据 result/detailed-screenshots/D05_edited_draft.png

对应链条：POST /api/orders 创建，PUT /api/orders/该编号修改，GET 读取明细。该组截图把输入数量、计算公式和数据库返回结果放在同一流程中，支持多商品与明细修改要求。

W07 W08 店员确认订单并实际扣减库存

步骤 W07 clerk 找到该草稿，点击确认，再点击查看。观察结果：同一订单状态已完成；数量、成交价、22.00 元总额保留。与 W05 为同一订单，完成时仍按原草稿单价计算 22.00 元；确认由 clerk 执行。



订单明细 WEB-acd02e7cd4894de4ab4c

状态 已完成 · 合计 ¥22.00

行号	饮品	杯数	成交单价	金额
1	珍珠奶茶	3	¥6.00	¥18.00
2	冰鲜柠檬水	1	¥4.00	¥4.00

图 12 W07 店员确认后的同一订单

原始证据 result/detailed-screenshots/D07_confirmed.png

步骤 W08 店员确认后点击库存页。观察结果：D001 从 20 到 17、D003 从 5 到 4；D004 零库存显示缺货。对照 S01 原库存：D001 扣 3 杯，20−3=17；D003 扣 1 杯，5−1=4。店员页面只有查询，无盘点维护按钮。

编号	饮品	可售杯数	预警阈值	状态
D001	珍珠奶茶	17	10	充足
D002	满杯百香果	12	12	低库存
D003	冰鲜柠檬水	4	8	低库存
D004	椰果奶茶	0	6	缺货
D005	柠檬红茶	18	4	充足
D006	茉莉绿茶	9	5	充足

图 13 W08 实际扣库与库存预警

原始证据 result/detailed-screenshots/D08_deducted_stock.png

W06 W09 顾客订单隔离与完成状态同步

步骤 W06 guest 登录后进入订单中心。观察结果：订单列表暂无记录，看不到 customer 刚建立的订单。这是在 customer 草稿已经存在时查看 guest 列表；同库不同会话，看不到他人订单。



图 14 W06 不同顾客订单隔离

原始证据 result/detailed-screenshots/D06_guest_isolation.png

步骤 W09 customer 重新进入订单中心。观察结果：本人订单已完成，顾客不再有编辑或删除草稿按钮。顾客重新查询后可看到店员处理结果，完成单不再显示草稿编辑与删除按钮。



图 15 W09 顾客看到店员确认结果

原始证据 result/detailed-screenshots/D09_customer_completed.png

查询过程按实际登录账号过滤，而不是让页面自行隐藏他人行。下面 SQL 的@ActorId 由可信会话经 DAO 传入；顾客条件限制 CreatedByUserId，员工可查全部。猜测他人编号后读取或写入的反例另在集成测试中验证。

```
SELECT o.OrderNo,o.OrderedAt,o.Status,o.MemberCode,o.EmployeeCode,o.CreatedByUserId,
 CONVERT(decimal(18,2),COALESCE(SUM(l.Quantity*l.UnitPrice),0)) AS Total
FROM dbo.ShopOrder o LEFT JOIN dbo.OrderLine l ON o.OrderNo=l.OrderNo
WHERE (@OrderNo IS NULL OR o.OrderNo=@OrderNo) AND (@Role<>'customer' OR o.CreatedByUserId=@ActorId)
GROUP BY o.OrderNo,o.OrderedAt,o.Status,o.MemberCode,o.EmployeeCode,o.CreatedByUserId
ORDER BY o.OrderedAt DESC,o.OrderNo;
```

代码来源 sql/application.sql 第 19 至 24 行；正文仅节选，完整文件随包提供。

空列表图只说明该次账号查询结果为空；结合已存在的 customer 订单、不同会话的请求记录和跨顾客拒绝用例，才支持订单隔离的结论。

W19 W20 散客草稿的创建与必要删除

步骤 W19 guest 直接保存默认 D001 一杯草稿。观察结果：自己的新订单为待确认、合计 6.00 元；会员关联由后端确定为空。guest 没有绑定会员，仍可建立自己的新草稿；会员 NULL 由数据库测试核对，截图本身只展示数量和金额。



图 16 W19 散客也能建立草稿

原始证据 result/detailed-screenshots/D19_guest_draft.png

步骤 W20 guest 确认删除自己的草稿。观察结果：本人订单列表再次暂无记录。DELETE 后列表为空；集成测试另检查头和明细都已删除。



图 17 W20 删除草稿及其明细

原始证据 result/detailed-screenshots/D20_deleted_draft.png

会员 customer 与散客 guest 在应用中都属于 customer 角色，区别是账号是否绑定 MemberCode。服务与过程根据账号确定会员，不信任顾客提交的任意会员号；因此不能通过修改 JSON 把草稿挂到别人名下。

删除仅针对尚未确认的 DRAFT。在同一事务先删明细再删头，避免孤立数据；完成单则保留，通过取消记录状态和库存回补。该区别对应课程要求中的必要删除，并不允许为展示 CRUD 而随意抹去已完成交易。

W10 管理员新增商品与修改菜单价格

步骤 W10 admin 新增 UI_TEST 验收测试茶，3.50 元；编辑为 4.00 元。观察结果：商品卡片显示 UI_TEST 及 4.00 元。新增字段为编号 UI_TEST、名称验收测试茶、类别测试饮品、价格 3.50、阈值 2；再次编辑价格 4.00。图展示修改完成后的卡片。



图 18 W10 新增商品与修改价格

原始证据 result/detailed-screenshots/D10_product.png

操作过程：在商品维护表单输入上述值并保存，等待新增卡片；点击该卡片的编辑，在回填表单改价并保存。DAO 新增时同事务建立零库存；修改只更新商品行。POST 与 PUT 均实际返回 200，见 detailed-browser.json 的 UI_TEST 请求。

菜单当前价与历史成交价是不同事实。本图只证明菜单维护成功；历史订单不受改价影响的结论由 S02 第 09 项及集成测试提供。

W12 管理员盘点库存为八杯

步骤 W12 UI_TEST 盘点杯数输入 8，点击保存库存。观察结果：表格显示 UI_TEST 为 8 杯。在库存维护表单输入商品编号 UI_TEST、quantity=8 并保存，等待表格返回新值。



图 19 W12 管理员盘点库存

原始证据 result/detailed-screenshots/D12_inventory_eight.png

读图定位 UI_TEST 这一行：可售杯数 8、预警阈值 2、状态充足。管理员比店员多出盘点及删除记录操作。盘点修改的是当前可售杯数，不代表一笔销售，也不会更改订单历史。

对应接口 PUT /api/inventory/UI_TEST，Service 要求 admin 及 0 至 1000000 的整数；DAO 锁定商品后更新或新增库存，最后统一提交。负数、小数、布尔值和不存在商品的处理见集成测试。

W13 零库存删除和重新登记

步骤 W13 先把 UI_TEST 设为 0 并等待新值，再确认删除记录。观察结果：商品仍存在，杯数为未登记，证明只删除库存行。先将杯数改为 0 并等待表格确实显示 0，之后点击删除记录并确认。图是删除后的查询结果。



图 20 W13 删除零库存记录

原始证据 result/detailed-screenshots/D13_deleted_inventory.png

图中 UI_TEST 商品行仍存在，但杯数和状态显示未登记。这是 Product 左连接 Inventory 的结果，说明删除的是库存记录，未把商品一并删掉。随后仍用同一库存表单保存 0 杯，可重新建立记录；该 PUT 请求已保存在运行记录。

同名成功通知可能仍来自上一次保存，因此自动化必须等待表格数值更新再删除，不能只等“库存已保存”。本轮已按实际行值等待，避免把测试脚本竞态误当作应用缺陷。

W14 删除无订单引用商品后的结果

步骤 W14 零库存重建后删除 UI_TEST 商品。观察结果：商品卡片恢复为原六种；UI_TEST 不存在。零库存记录重建后返回菜单，删除 UI_TEST 并确认；等待商品已删除通知和菜单刷新。



图 21 W14 删除没有订单引用的商品

原始证据 result/detailed-screenshots/D14_deleted_product.png

图中只剩原六种商品，测试饮品不再存在。此操作只允许无订单引用且库存为 0 的商品；如果商品已被订单引用，FK 将拒绝硬删除，事务回滚，管理员可改为下架。被引用删除的反例由集成测试真实执行，不以当前成功删除图证明。

W15 订单确认后的销售统计

步骤 W15 管理员进入销量页。观察结果：D001 销量 4+3=7，销售额 24+18=42；D003 销量 5+1=6，金额 24。在取消订单之前查询，所以该 22 元完成单应当计入销量。

编号	饮品	销量 杯	销售额 元
D001	珍珠奶茶	7	42.00
D002	满杯百香果	3	21.00
D003	冰鲜柠檬水	6	24.00
D004	椰果奶茶	3	21.00
D005	柠檬红茶	0	0.00
D006	茉莉绿茶	0	0.00

图 22 W15 完成订单计入销售统计

原始证据 result/detailed-screenshots/D15_sales.png

商品或合计	S01 完成销售基线	本轮新增完成销售	本步骤实际预期
D001	4 杯 24.00 元	3 杯 18.00 元	7 杯 42.00 元
D003	5 杯 20.00 元	1 杯 4.00 元	6 杯 24.00 元
全部饮品	15 杯 86.00 元	4 杯 22.00 元	19 杯 108.00 元

表中合计由图内六行求和得到，页面没有单独展示总额卡片。销售查询仅统计 COMPLETED，保留零销量商品；取消后的销售是否退出统计由集成断言核对。

W17 取消完成单后订单历史仍保留

步骤 W17 clerk 取消该 22.00 元新系统完成单。观察结果：订单列表保留同一订单，状态已取消。找到 W04 至 W07 同一 WEB 编号执行取消。订单金额仍为 22.00 元，状态改为已取消，而不是直接删除记录。

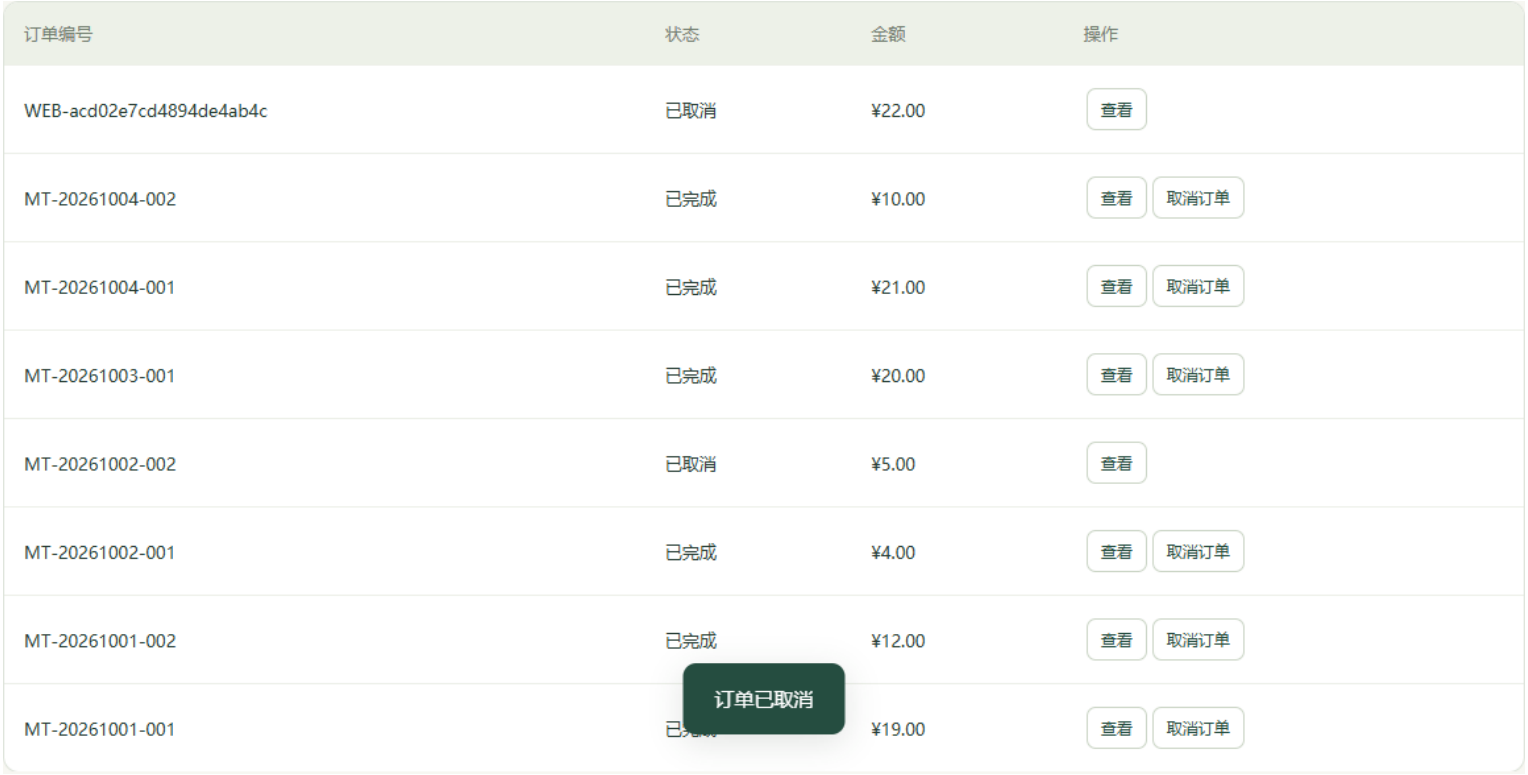


图 23 W17 取消后保留订单历史

原始证据 result/detailed-screenshots/D17_cancelled.png

原七笔历史订单同时显示在店员列表中，其中原本就有一笔取消单。本步骤应定位本轮新建 WEB 编号这一行，避免把历史取消单当成取消动作成功的证据。下一图另查实际库存，核对回补结果。

W18 取消后库存回补的实际数值

步骤 W18 取消完成单后重新查询库存。观察结果：D001 恢复 20、D003 恢复 5；与 S01 基线一致。在取消接口返回成功后重新发起库存查询，等待 D001=20 且 D003=5 再截图。



编号	饮品	可售杯数	预警阈值	状态
D001	珍珠奶茶	20	10	充足
D002	满杯百香果	12	12	低库存
D003	冰鲜柠檬水	5	8	低库存
D004	椰果奶茶	0	6	缺货
D005	柠檬红茶	18	4	充足
D006	茉莉绿茶	9	5	充足

图 24 W18 取消后的实际库存恢复

原始证据 result/detailed-screenshots/D18_restored_stock.png

商品	确认前 S01	确认后 W08	取消后 W18
D001	20	17	20
D003	5	4	5

前后对照支持本轮完成单回补了实际扣减数量；“重复取消不重复回补”及“零库存行被删除后取消仍能重建”的边界结论，由 T01 对应集成用例再次验证。历史 StockApplied=0 的完成单不可套用本流程。

W11 W16 错误反馈和实际请求的对应

步骤 W11 新增表单再次输入相同 UI_TEST 编号并提交。观察结果：记录重复或已被引用；409；界面附请求编号。数据库唯一约束真实产生冲突；API 为 409 CONSTRAINT_CONFLICT，未重复插入商品。

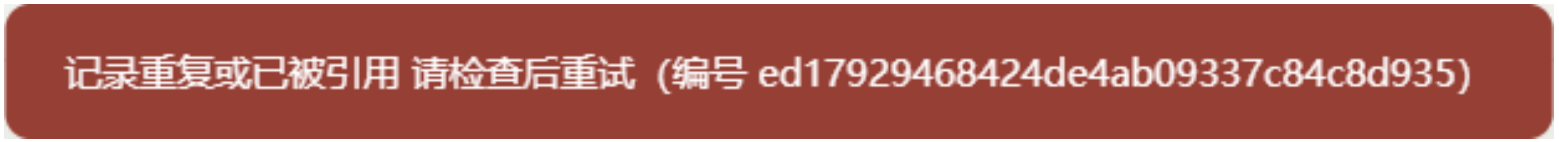


图 25 W11 真实唯一约束冲突

原始证据 result/detailed-screenshots/D11_constraint.png

步骤 W16 仅将隔离验收服务认证连接临时改为本机未监听端口，再访问库存。观察结果：系统暂时无法连接；503；不展示 SQL、地址或堆栈。临时故障只作用于本次隔离服务；随即恢复原连接配置。API 为 503 DB_UNAVAILABLE。

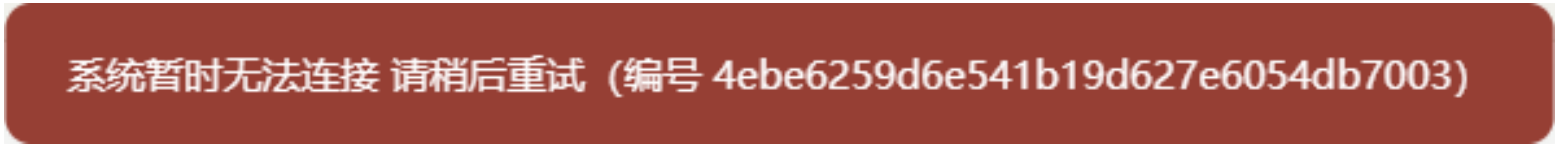


图 26 W16 真实连接失败的友好反馈

原始证据 result/detailed-screenshots/D16_connection.png

步骤	HTTP 与错误码	实际请求编号
W11	409 CONSTRAINT_CONFLICT	ed17929468424de4ab09337c84c8d935
W16	503 DB_UNAVAILABLE	4ebe6259d6e541b19d627e6054db7003

图中的编号、上表及公开 JSONL 日志一一对应。两张通知图只证明用户看到的提示；原始请求记录和日志进一步证明错误来自实际请求。界面均未包含 SQL、服务器地址、文件路径或异常堆栈。

W21 移动尺寸页面的实际检查

步骤 W21：使用真实 Edge 引擎创建 390×844 视口，以移动布局选项打开登录页。执行 document.documentElement.scrollWidth <= window.innerWidth 断言通过，再截图。原图为完整页面，页面内容可以纵向滚动。



图 27 W21 390 像素宽移动布局的登录页

原始证据 result/detailed-screenshots/D21_mobile.png

图中标题、输入框和登录按钮未横向溢出；本项只验证移动尺寸登录页面，不宣称已覆盖全部手机型号或移动端业务页面。

测试设计与关键反例的输入和断言

测试不只检查 HTTP 成功，还直接查询数据库确认状态、行数、库存和金额。conftest 强制加载 private/test.env 且库名必须以_Test 结尾，每例前后恢复 Baseline；因此输入相同、预期确定，各用例不会互相依赖脏数据。

用例场景	实际输入或触发	预期与实际断言
账号与会话	四角色登录退出；错误密码；停用当前账号	正常 200；错误 401；停用会话下次请求拒绝
后端越权	customer/clerk 直接请求商品或库存修改	403；绕过按钮不可获得权限
顾客隔离	A 创建草稿后 B 查询、修改、删除该编号	404 或拒绝；A 数据保留
SQL 与 HTML 输入	登录注入、搜索注入、特殊商品名称	无绕过；查询按值绑定；文本不当 HTML 执行
库存参数	负数、小数、布尔值及非法 JSON	400；原库存不变
引用与唯一	重复编号；被引用商品删除；非法会员	409 或业务拒绝；不留下部分头或明细
缺货回滚	订单前项 D001 可扣，后项 D004 库存 0	409；前项已执行扣减回滚，状态仍草稿
并发不超卖	D001 库存 20；两单并发各购买 15 杯	一单 200、一单 409；最终库存 5，非负
重复动作	同一订单确认两次、取消两次	首个合法动作成功，第二次拒绝；不重复扣回
历史价格与库存	保存报价后改菜单价；历史单取消	原报价确认；历史开库单禁止回补
实际数据库权限	四应用角色查询有效身份并尝试越权 SQL	sysadmin=0、db_owner=0；DDL、敏感读写拒绝
连接与事务异常	真实不可达连接；注入 40001 异常	503 且友好错误；日志包含对应 request_id

下一组三张 T01 截图连续覆盖完整详细输出，共 43 个用例。该数包含参数化展开，不应把测试函数个数当作测试例数。命令、实际耗时和 XML 均来自本轮重新执行。

T01 集成测试实际输出 1

步骤 T01：执行 .venv\Scripts\python.exe -m pytest stage2/tests -v，并生成独立 JUnit XML。

本段包括五项复核用例、迁移基线和多角色登录。PASSED 只有断言完成后才出现；不是手工把测试名排成通过列表。

真实SQL Server集成测试 实际输出 1/3

这是执行输出的只读核验页，非SSMS或终端原生界面。原文逐行保留。

数据库 MilkTeaShopV10_Acceptance_Test　退出码 0　执行开始 UTC 2026-11-08T10:20:54.372285+00:00

执行命令 D:\Homework\数据库\campus-shop-db-lab\.venv\Scripts\python.exe -m pytest stage2/tests -v --junitxml=stage2/result/detailed-runs/pytest.xml

```
============================= test session starts =============================
platform win32 -- Python 3.14.2, pytest-8.4.2, pluggy-1.7.0 -- D:\\Homework\\\u6570\u636e\u5e93\\campus-shop-db-
lab\\.venv\\Scripts\\python.exe
cachedir: .pytest_cache
rootdir: D:\\Homework\\\u6570\u636e\u5e93\\campus-shop-db-lab
collecting ... collected 43 items

stage2/tests/test_course_audit.py::test_large_valid_order_total_remains_readable PASSED [  2%]
stage2/tests/test_course_audit.py::test_cancel_restores_stock_after_zero_inventory_record_deleted PASSED [  4%]
stage2/tests/test_course_audit.py::test_non_ascii_csrf_is_rejected_as_input_not_system_error PASSED [  6%]
stage2/tests/test_course_audit.py::test_connection_failure_log_preserves_last_authenticated_role PASSED [  9%]
stage2/tests/test_course_audit.py::test_menu_search_does_not_remove_other_products_from_draft_editor PASSED [ 11%]
stage2/tests/test_integration.py::test_baseline_and_migration PASSED     [ 13%]
stage2/tests/test_integration.py::test_login_all_roles_and_logout[admin] PASSED [ 16%]
stage2/tests/test_integration.py::test_login_all_roles_and_logout[clerk] PASSED [ 18%]
stage2/tests/test_integration.py::test_login_all_roles_and_logout[customer] PASSED [ 20%]
stage2/tests/test_integration.py::test_login_all_roles_and_logout[guest] PASSED [ 23%]
stage2/tests/test_integration.py::test_unauthenticated_and_csrf PASSED   [ 25%]
stage2/tests/test_integration.py::test_bad_credentials_and_login_injection[admin' OR 1=1--] PASSED [ 27%]
```

原始输出 result/detailed-runs/T01_pytest.txt　SHA256 1936a5f82868328f81ee28ce5be53be8467b7e552c13ab338b871a99de30ae2c
本页为原输出第 1 至 18 行

图 28 T01_pytest 真实 SQL Server 集成测试 输出第 1 至 18 行

原始证据 result/detailed-screenshots/T01_pytest-1.png

T01 集成测试实际输出 2

步骤 T01：执行 .venv\Scripts\python.exe -m pytest stage2/tests -v，并生成独立 JUnit XML。

本段继续覆盖认证、越权、商品库存 CRUD、无效参数、注入和约束、顾客所有权等。参数化用例逐个列出。

真实SQL Server集成测试 实际输出 2/3

这是执行输出的只读核验页，非SSMS或终端原生界面。原文逐行保留。

数据库 MilkTeaShopV10_Acceptance_Test　退出码 0　执行开始 UTC 2026-11-08T10:20:54.372285+00:00

执行命令 D:\Homework\数据库\campus-shop-db-lab\.venv\Scripts\python.exe -m pytest stage2/tests -v --junitxml=stage2/result/detailed-runs/pytest.xml

```
stage2/tests/test_integration.py::test_bad_credentials_and_login_injection[admin] PASSED [ 30%]
stage2/tests/test_integration.py::test_bad_credentials_and_login_injection[unknown] PASSED [ 32%]
stage2/tests/test_integration.py::test_backend_role_denial[clerk] PASSED [ 34%]
stage2/tests/test_integration.py::test_backend_role_denial[customer] PASSED [ 37%]
stage2/tests/test_integration.py::test_product_and_inventory_crud PASSED [ 39%]
stage2/tests/test_integration.py::test_invalid_inventory_parameters[-1] PASSED [ 41%]
stage2/tests/test_integration.py::test_invalid_inventory_parameters[1.5] PASSED [ 44%]
stage2/tests/test_integration.py::test_invalid_inventory_parameters[True] PASSED [ 46%]
stage2/tests/test_integration.py::test_invalid_inventory_parameters[3] PASSED [ 48%]
stage2/tests/test_integration.py::test_invalid_inventory_parameters[None] PASSED [ 51%]
stage2/tests/test_integration.py::test_invalid_inventory_parameters[1000001] PASSED [ 53%]
stage2/tests/test_integration.py::test_missing_and_invalid_json PASSED   [ 55%]
stage2/tests/test_integration.py::test_search_parameterization_and_output_escaping PASSED [ 58%]
stage2/tests/test_integration.py::test_constraint_duplicate_and_referenced_delete PASSED [ 60%]
stage2/tests/test_integration.py::test_customer_full_draft_crud_and_ownership PASSED [ 62%]
stage2/tests/test_integration.py::test_guest_and_customer_cannot_forge_member PASSED [ 65%]
stage2/tests/test_integration.py::test_submit_stock_prices_cancel_and_repeat PASSED [ 67%]
stage2/tests/test_integration.py::test_short_stock_rolls_back_earlier_product_deduction PASSED [ 69%]
```

原始输出 result/detailed-runs/T01_pytest.txt　SHA256 1936a5f82868328f81ee28ce5be53be8467b7e552c13ab338b871a99de30ae2c
本页为原输出第 19 至 36 行

图 29　T01_pytest 真实 SQL Server 集成测试 输出第 19 至 36 行

原始证据 result/detailed-screenshots/T01_pytest-2.png

T01 集成测试实际输出 3

步骤 T01：执行 .venv\Scripts\python.exe -m pytest stage2/tests -v，并生成独立 JUnit XML。

本段包括订单事务、缺货回滚、非法会员、并发、真实应用角色限制、连接错误和停用会话。末尾给出 43 passed 及实际耗时；XML 统计 tests=43、failures=0、errors=0。

真实SQL Server集成测试 实际输出 3/3

这是执行输出的只读核验页，非SSMS或终端原生界面。原文逐行保留。

数据库 MilkTeaShopV10_Acceptance_Test　退出码 0　执行开始 UTC 2026-11-08T10:20:54.372285+00:00

执行命令 D:\Homework\数据库\campus-shop-db-lab\.venv\Scripts\python.exe -m pytest stage2/tests -v --junitxml=stage2/result/detailed-runs/pytest.xml

```
stage2/tests/test_integration.py::test_invalid_member_whole_order_rollback PASSED [ 72%]
stage2/tests/test_integration.py::test_inactive_product_and_empty_order PASSED [ 74%]
stage2/tests/test_integration.py::test_historical_order_cancellation_is_blocked PASSED [ 76%]
stage2/tests/test_integration.py::test_concurrent_stock_no_overselling PASSED [ 79%]
stage2/tests/test_integration.py::test_actual_restricted_database_accounts[auth] PASSED [ 81%]
stage2/tests/test_integration.py::test_actual_restricted_database_accounts[admin] PASSED [ 83%]
stage2/tests/test_integration.py::test_actual_restricted_database_accounts[clerk] PASSED [ 86%]
stage2/tests/test_integration.py::test_actual_restricted_database_accounts[customer] PASSED [ 88%]
stage2/tests/test_integration.py::test_direct_database_product_and_stock_mutation_denied[clerk] PASSED [ 90%]
stage2/tests/test_integration.py::test_direct_database_product_and_stock_mutation_denied[customer] PASSED [ 93%]
stage2/tests/test_integration.py::test_database_connection_failure_is_sanitized PASSED [ 95%]
stage2/tests/test_integration.py::test_transaction_failure_is_sanitized_and_logged PASSED [ 97%]
stage2/tests/test_integration.py::test_deactivated_account_session_is_rejected PASSED [100%]

- generated xml file: D:\\Homework\\\u6570\u636e\u5e93\\campus-shop-db-lab\\stage2\\result\\detailed-runs\\pytest.xml -
============================== 43 passed in 44.84s ==============================
```

原始输出 result/detailed-runs/T01_pytest.txt　SHA256 1936a5f82868328f81ee28ce5be53be8467b7e552c13ab338b871a99de30ae2c
本页为原输出第 37 至 52 行

图 30 T01_pytest 真实 SQL Server 集成测试 输出第 37 至 52 行

原始证据 result/detailed-screenshots/T01_pytest-3.png

复核发现的问题 修复和再次验证

初轮通过后，再按交叉操作和边界条件复核，实际复现五个缺陷。五个新增测试在旧实现均失败；修复后五个均通过，再执行完整 43 例和真实浏览器流程。以下保留问题及影响，避免只给“全部通过”而缺少改进过程。

问题和触发	修改前表现	最终处理和验证
搜索 D001 后编辑含 D003 草稿	下拉框丢失 D003 选项	筛选展示与完整目录分开；修改 3 杯仍为 12.00 元
售空后删零库存行，再取消完成单	返回成功但未恢复库存行	锁定商品并补建缺失库存；恢复 2 杯；重复取消拒绝
合法单价上限与大数量	保存成功，查询总额溢出 503	汇总转 decimal(18,2)；19999999998.00 可读取
非 ASCII 错误 CSRF 头 é	字符串比较触发 500	UTF8 字节恒定时间比较；返回 403 CSRF_INVALID
已登录时身份复查遇连接失败	日志角色错误记录 anonymous	诊断保留最后认证角色；授权仍依赖实时数据库身份

另外在 ER 中补全联系名称并注明 v0.1 的最低一条明细仅为业务规则；规范化证明没有把电话可空唯一误当作全关系候选码。自动化脚本也增加库存行值等待，避免同文字通知导致竞态。证据见 course-audit.json 及 course-review.md。

失败证据和成功证据的边界

旧版本五项失败的摘要、测试文件和证据摘要值保留在 course-audit.json；本轮 T01 截图展示修复后实际结果。SQL 错误 229/547、真实连接不可达和并发争抢均为实际 SQL Server 执行；事务错误 40001 分类使用故障注入，不能称为已自然发生并复现真实死锁。

演示截图中库存确认和恢复用一笔可售的 22 元订单完成；缺货回滚、重复取消等反例由独立集成用例证明。报告明确每张图能证明什么，避免把成功界面扩张为所有边界都正确。

提交包复现与测试后数据核验

此前已将提交包解压到独立中文路径，新建虚拟环境并从 requirements 安装，再用包内 SQL 建立新数据库。复制后的 SQL 回归 13 项、集成 43 例、浏览器 18 项以及 BAT 入口四账号登录均通过；记录位于 result/package-replay.json。本版扩写文档和采集工具，应用和核心 SQL 保持与该已执行副本一致，最终 ZIP 另校验全部文件摘要。

空环境复现的主要命令

在解压后包含 sql、stage2 和启动 BAT 的根目录执行：

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r stage2/requirements-dev.txt
.\.venv\Scripts\python.exe -m stage2.tools.setup --from-scripts --database MilkTeaShopV10_Demo
.\.venv\Scripts\python.exe -m stage2
```

目标库或配置已存在时脚本拒绝覆盖，应换新库名及--env-file。已有 v0.1 时不使用--from-scripts，脚本先 COPY_ONLY 备份，再还原到新库后迁移。随机密码保存在初始化机器自己的 private 目录，不能从公共报告寻找通用密码。

测试后的核验

本轮浏览器结束后再运行基线测试，清理本轮 WEB 订单及测试商品并恢复_Test 库库存，随后核对原库六表与演示库的原字段。原库和演示库不执行测试清理。收尾证据见 result/detailed-cleanup.txt 和 result/detailed-final-validation.json。

复现检查	实际验收记录	本版范围
独立新库和新环境	package-replay.json 及 package-pytest.xml	此前复制包真实执行；最终检查核心文件摘要一致
本轮补采	detailed-browser.json	18 项检查、60 次 HTTP、21 个指定界面状态；无未捕获 JS 异常
本轮 SQL 与测试	detailed-runs/manifest.json	13 项 SQL、43 例集成重新执行；原始 stdout 和 XML 齐全
提交文件完整性	ZIP 内 SHA256.json	逐文件摘要、凭据排除、Word/PDF 和配图来源检查

三 实验总结与小组责任

本阶段将 v0.1 的六表样例扩展为有账号、草稿和库存事务的实际 Web 应用。ER 模型明确可选会员、订单明细和角色关联；FD 分析支持保留合理结构；迁移以独立副本和语义回归证明历史不被改写。页面操作与 API、DAO、数据库约束和事务有实际对应，结果由金额、杯数、状态和错误请求共同核对。

复核经验是把操作串起来：搜索会影响草稿编辑，商品上限会影响汇总，库存删除会影响取消。单个接口成功不能替代跨场景验证；截图也必须写清产生条件和观察对象。后续可扩展原料配方、支付退款和公网部署，但这些不在本阶段完成声明中。

成员	责任及课堂演示安排
袁远冬	需求、ER、迁移和仓库整合；演示基数、独立副本及迁移前后核验
陆乐天	FD、范式、数据字典与 DAO；演示无损分解、约束及 CRUD 回归
罗金浩	Web、角色与异常、测试证据与报告；演示 22 元订单全流程、回滚和请求追踪

上表是责任与讲解安排；本次可核验的代码、文档和运行主要由 AI 辅助形成。没有虚构三人分别手工开发的时长、讨论原话或个人验收签名。人工阅读、独立操作和签字应由本人完成并如实补充。AI 使用过程见 docs/ai_log.md，分工说明见 docs/contribution.md。

证据类型	读取方法
设计和迁移	docs/design.md、data-dictionary.md、migration-guide.md；可编辑 SVG 与 SQL
逐图索引	result/detailed-figure-index.md；按图号、步骤和原图定位
真实界面与请求	result/detailed-screenshots/、detailed-browser.json、detailed-request-log.jsonl
实际 SQL 及测试输出	result/detailed-runs/；manifest 记录命令、时间、退出码、行号和 SHA256
原数据与复核	before-after.json；course-audit.json；package-replay.json

课程依据：本地第五周、第六周、第 7 至 9 周任务讲解。技术参考：Flask 官方 Web 安全与会话文档；Microsoft Learn 的 sp_setapprole、应用程序角色及 SQL Server 表提示文档。课程没有给出明确评分量表，本报告据任务逐项验收，不推断教师分数。